

TinyML-Based Smart Traffic Light Control System

Abstract:

Traffic congestion has become a significant challenge in modern urban environments, leading to increased travel time, higher fuel consumption, and elevated levels of environmental pollution. Traditional traffic light systems operate on fixed time intervals and lack the ability to adapt to real-time traffic conditions, resulting in inefficient traffic management. This project proposes a TinyML-based smart traffic light control system that utilizes low-power edge computing for dynamic traffic regulation.

The system employs an ESP32-CAM module to capture real-time traffic images and integrates a lightweight machine learning model implemented using TensorFlow Lite for Microcontrollers. The model classifies vehicle density into three categories: low, medium, and high. Based on this classification, the system dynamically adjusts traffic signal timings to optimize traffic flow and minimize congestion.

By performing on-device inference, the proposed system ensures low latency, enhanced data privacy, and reduced power consumption, eliminating the dependency on cloud-based processing. This approach effectively combines TinyML, embedded systems, and intelligent sensing to deliver a scalable, cost-effective, and efficient solution for adaptive traffic management in smart cities.

Keywords: TinyML, Smart Traffic Control, ESP32-CAM, Traffic Congestion, Edge Computing, TensorFlow Lite, Intelligent Transportation Systems

By

S.no	Name	Registration No	Signature
1.	Sarbeswar Dash	NDU202500369	Sarbeswar Dash
2.	Jyoti Verma	NDU202500370	Jyoti Verma
3.	A. Architha	NDU202500371	A. Architha

Under the Guidance:

Dr. Arun T Nair

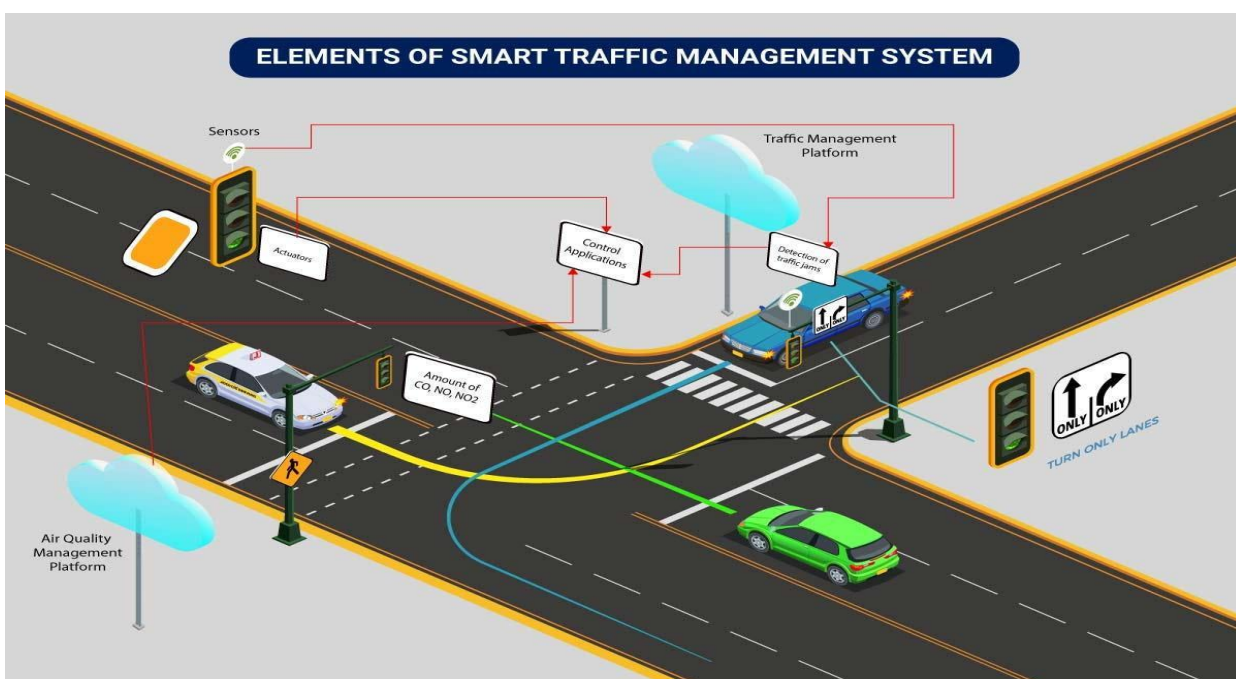
Introduction

Traffic congestion has become one of the most significant challenges in modern urban environments due to the rapid increase in the number of vehicles and the expansion of road networks. As urban populations continue to grow, the demand for efficient transportation systems has also increased. However, existing traffic management systems are often unable to cope with this demand, resulting in frequent traffic jams, longer travel times, and increased commuter frustration.

Conventional traffic signal systems operate on fixed time intervals that are predefined and do not change according to real-time traffic conditions. These systems assume uniform traffic distribution across all directions, which is rarely the case in real-world scenarios. As a result, vehicles on less congested roads are forced to wait unnecessarily, while heavily congested lanes do not receive sufficient green signal duration. This imbalance leads to inefficient utilization of road infrastructure.

Another major drawback of traditional systems is the lack of real-time monitoring and decision-making capabilities. Since these systems do not collect or analyze live traffic data, they are unable to respond dynamically to sudden changes in traffic patterns such as peak hours, accidents, or unexpected congestion. This not only increases waiting time but also contributes to excessive fuel consumption and higher levels of air pollution due to prolonged engine idling.

With the advancement of embedded systems, Internet of Things (IoT), and artificial intelligence, there is a growing need to develop intelligent traffic control systems that can adapt to real-time conditions. In recent years, machine learning techniques have been widely used for traffic analysis, particularly in the field of computer vision, where images or video streams are used to detect and classify vehicles. However, many of these solutions rely on cloud-based processing, which introduces latency, requires continuous internet connectivity, and raises concerns related to data privacy

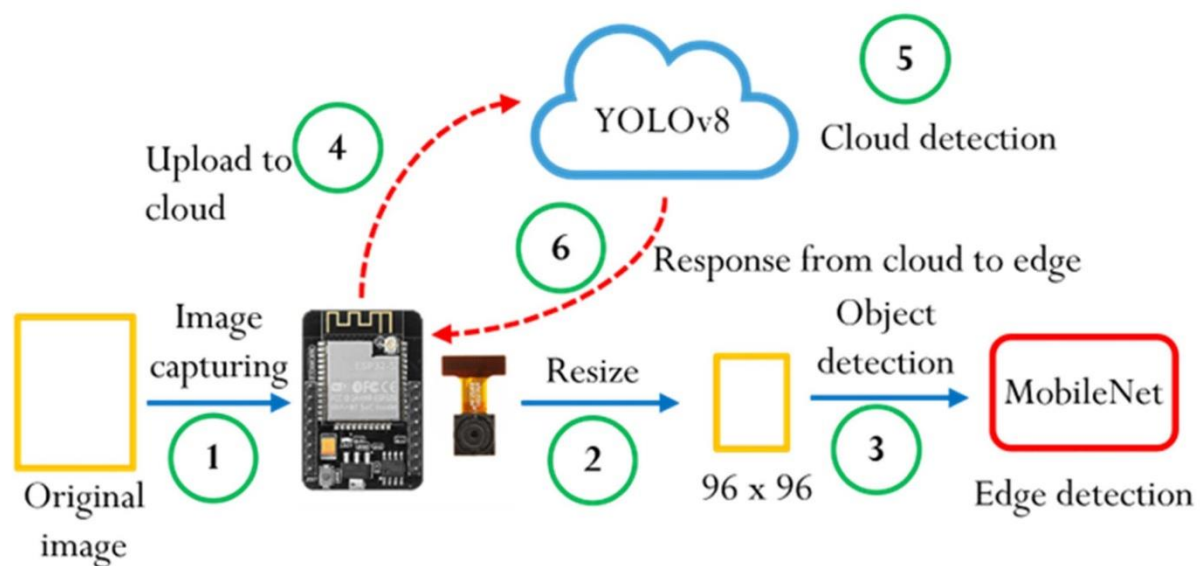


To overcome these limitations, TinyML has emerged as an efficient solution that enables machine learning models to run directly on microcontrollers. TinyML allows real-time data processing at the edge, reducing dependency on cloud infrastructure and ensuring faster response times. It is particularly suitable for low-power embedded devices, making it an ideal choice for smart traffic management applications.

In this project, a TinyML-based smart traffic light control system is proposed using the ESP32-CAM for real-time image acquisition and processing. The system utilizes a lightweight Convolutional Neural Network (CNN) model developed using Edge Impulse, which classifies traffic density into three categories: low, medium, and high. The classification is performed directly on the device, enabling fast and efficient decision-making.

Once the traffic density is determined, the result is transmitted wirelessly to an ESP32 DevKit, which acts as the traffic signal controller. Based on the received data, the system dynamically adjusts the traffic signal, prioritizing lanes with higher vehicle density. This ensures better traffic flow, reduces congestion, and improves overall transportation efficiency.

Furthermore, the proposed system is cost-effective, scalable, and easy to deploy, making it suitable for smart city applications. By integrating TinyML with embedded systems and wireless communication, this project demonstrates a practical approach to real-time traffic management using edge intelligence.

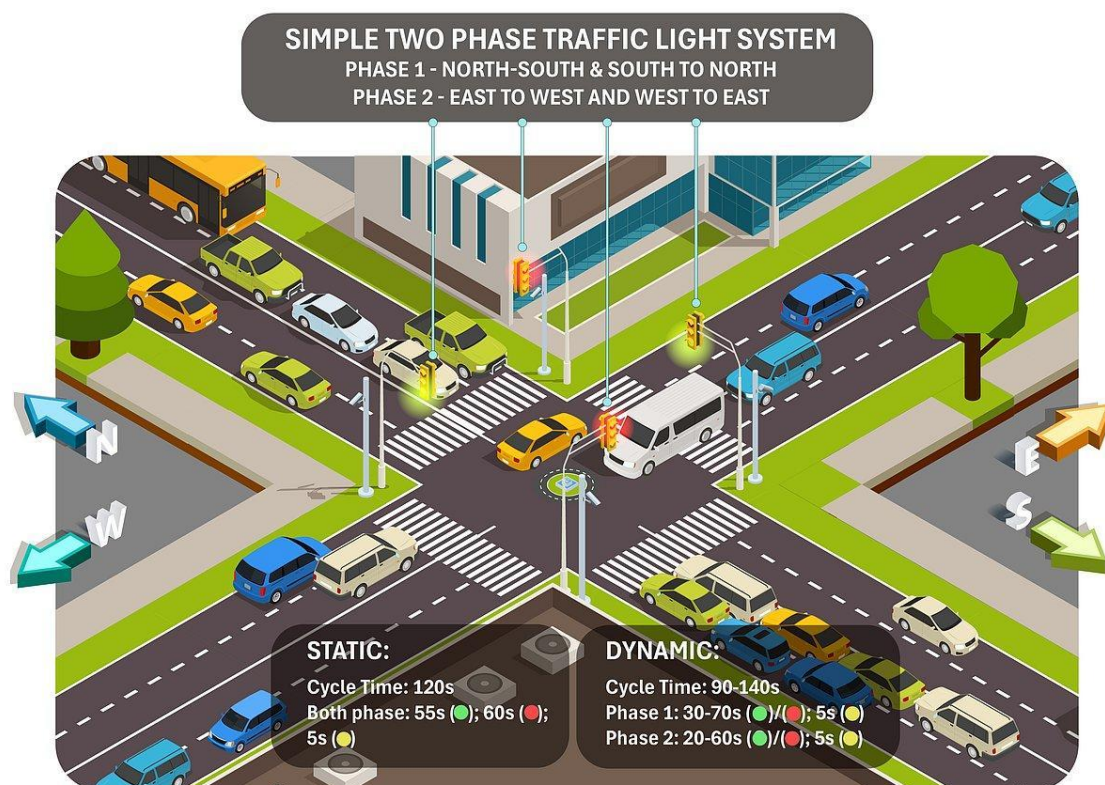


Problem Statement

Traffic congestion has become a critical issue in modern urban environments due to the rapid growth in population and the increasing number of vehicles on the road. Existing traffic management systems are largely based on fixed-time signal control, where traffic lights operate according to predefined time intervals regardless of actual traffic conditions. This approach fails to address real-time variations in vehicle density at intersections.

In many situations, one lane may experience heavy traffic congestion while another lane remains relatively empty. However, due to fixed signal timing, vehicles in low-density lanes are forced to wait unnecessarily, while high-density lanes do not receive adequate green signal duration. This imbalance leads to inefficient traffic flow and increased delays.

Another major limitation of traditional traffic systems is the lack of real-time monitoring and intelligent decision-making. These systems do not utilize live traffic data, making them incapable of adapting to dynamic changes such as peak hours, accidents, or sudden increases in vehicle volume. As a result, traffic congestion continues to worsen, especially in densely populated urban areas.



The inefficiency of conventional traffic control systems also contributes to higher fuel consumption and environmental pollution. Vehicles stuck in traffic consume more fuel and emit harmful gases, negatively impacting both the economy and the environment. Additionally, prolonged waiting times reduce overall transportation efficiency and cause inconvenience to commuters.

Therefore, there is a strong need for an intelligent, adaptive, and automated traffic control system that can monitor traffic conditions in real time and make decisions accordingly. Such a system should be capable of analyzing traffic density, classifying it into meaningful categories, and dynamically adjusting signal timing to improve traffic flow.

The proposed solution addresses these challenges by integrating TinyML-based traffic density classification with embedded systems, enabling real-time and efficient traffic signal control without reliance on cloud infrastructure.

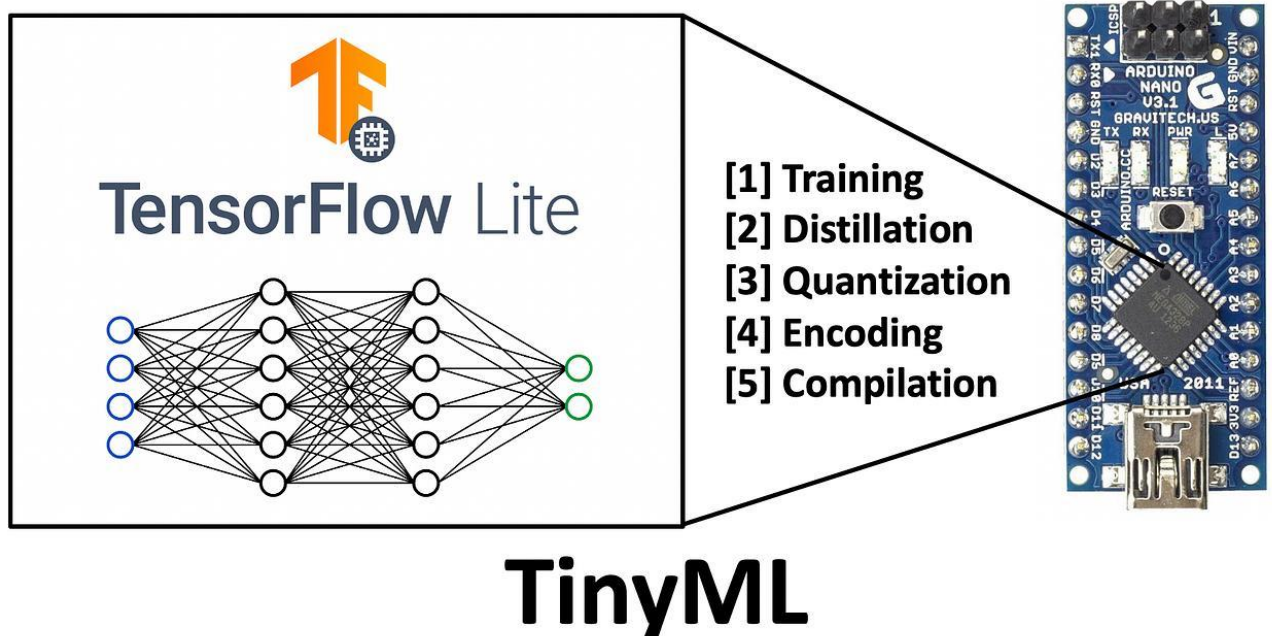
Objectives

The primary objective of this project is to design and develop an intelligent traffic light control system that can adapt dynamically to real-time traffic conditions using embedded systems and TinyML. The system aims to overcome the limitations of conventional fixed-time traffic signals by introducing automation and real-time decision-making capabilities.

One of the key objectives is to capture real-time traffic images using an ESP32-CAM and analyze them directly on the device. This eliminates the need for external processing systems and enables faster response times. The captured images are processed using a lightweight machine learning model developed through Edge Impulse, which allows efficient deployment on resource-constrained microcontrollers.

Another important objective is to classify traffic density into three distinct categories: low, medium, and high. This classification enables the system to understand the level of congestion at a particular intersection and make appropriate decisions regarding traffic signal control.

The project also aims to dynamically control traffic signals based on the classified traffic density. By adjusting the duration of the green signal according to real-time conditions, the system ensures smoother traffic flow and reduces unnecessary waiting time for vehicles. This contributes to improved road utilization and enhanced transportation efficiency.



In addition, the system is designed to enable wireless communication between devices using WiFi. The processed data is transmitted from the ESP32-CAM to an ESP32 DevKit, which acts as the central controller for traffic signal operation. This ensures seamless data transfer and real-time responsiveness.

Another objective of this project is to develop a low-cost and energy-efficient solution that can be easily deployed in real-world scenarios. By using affordable hardware components and on-device machine learning, the system minimizes infrastructure requirements and operational costs.

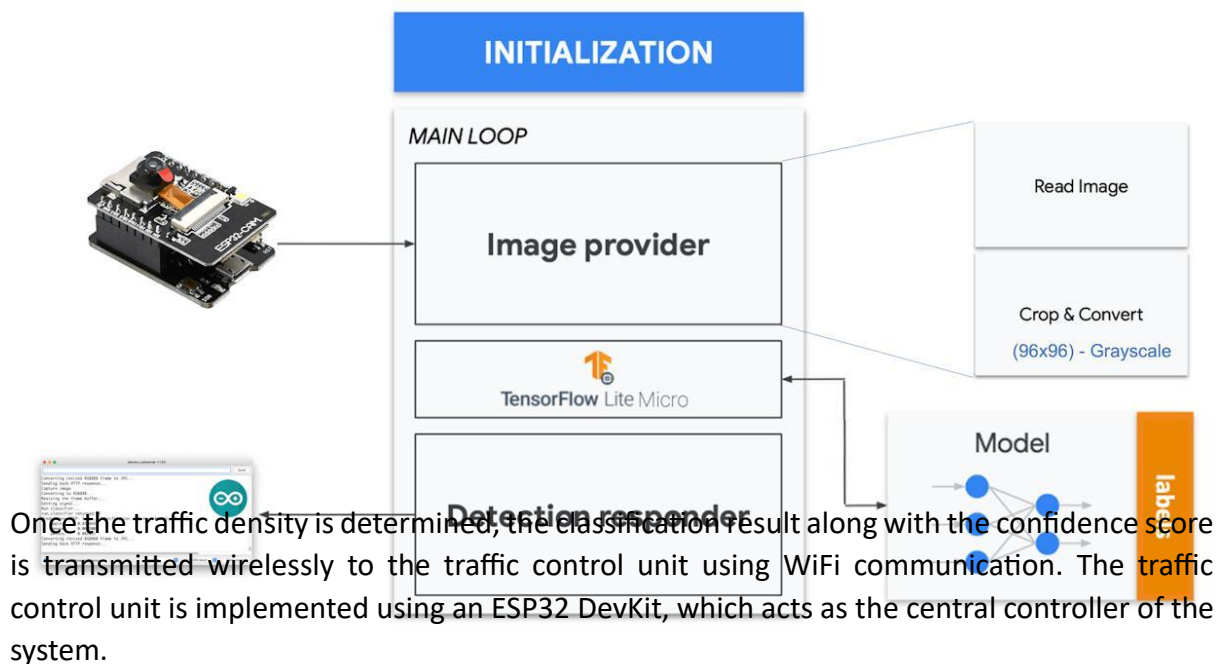
Finally, the project aims to demonstrate the practical application of TinyML in real-time embedded systems. It highlights how edge-based intelligence can be effectively used for smart city applications, particularly in traffic management systems....

Proposed System

The proposed system is an intelligent traffic light control solution that utilizes TinyML and embedded systems to dynamically manage traffic flow based on real-time vehicle density. Unlike traditional traffic systems that operate on fixed timing, this system adapts its behavior according to actual road conditions, thereby improving efficiency and reducing congestion.

The system is designed using two main components: an image acquisition and processing unit, and a traffic control unit. The image acquisition is performed using an ESP32-CAM, which captures real-time images of vehicles at a traffic intersection. This module is capable of both capturing and processing images, making it suitable for edge-based applications.

The captured images are processed using a lightweight Convolutional Neural Network (CNN) model developed using Edge Impulse. The model is trained to classify traffic density into three categories: low, medium, and high. Since the model runs directly on the ESP32-CAM, it eliminates the need for cloud-based processing, ensuring low latency and faster decision-making.



Based on the received data, the ESP32 DevKit dynamically controls the traffic signal. The duration of the green signal is adjusted according to the detected traffic density. For example, a higher traffic density results in a longer green signal duration, allowing more vehicles to pass through the intersection efficiently.

The system follows a real-time workflow where image capture, processing, classification, and signal control occur continuously. This ensures that the traffic signals are always updated according to current road conditions. The use of TinyML enables efficient computation on low-power devices, making the system suitable for scalable and cost-effective deployment.

System Architecture

The system architecture of the proposed smart traffic light control system is designed to enable real-time traffic monitoring and adaptive signal control using TinyML and embedded systems. It consists of multiple interconnected modules that work together to capture, process, transmit, and act upon traffic data efficiently.

The architecture is divided into four major functional blocks: image acquisition, data processing, communication, and traffic control.

1. Image Acquisition Module

The image acquisition module is implemented using the ESP32-CAM. This module is responsible for capturing real-time images of vehicles at a traffic intersection. The camera is positioned to obtain a clear view of the road, enabling accurate detection of traffic density.

2. Data Processing Module (TinyML Inference)

The captured images are processed locally on the ESP32-CAM using a lightweight Convolutional Neural Network (CNN) model. The model is developed and trained using Edge Impulse and deployed on the microcontroller. The model performs real-time inference and classifies traffic density into three categories: low, medium, and high.

By performing inference on-device, the system reduces latency, avoids dependency on cloud infrastructure, and ensures faster response time.

3. Communication Module

Once the classification is completed, the ESP32-CAM transmits the result (density level and confidence score) to the traffic control unit using WiFi communication. HTTP-based communication is used to ensure reliable data transfer between devices.

This module ensures seamless interaction between the sender (ESP32-CAM) and the receiver (ESP32 DevKit), enabling real-time updates.

4. Traffic Control Module

The traffic control module is implemented using the ESP32 DevKit. It receives the traffic density data from the ESP32-CAM and processes it to control the traffic signals.

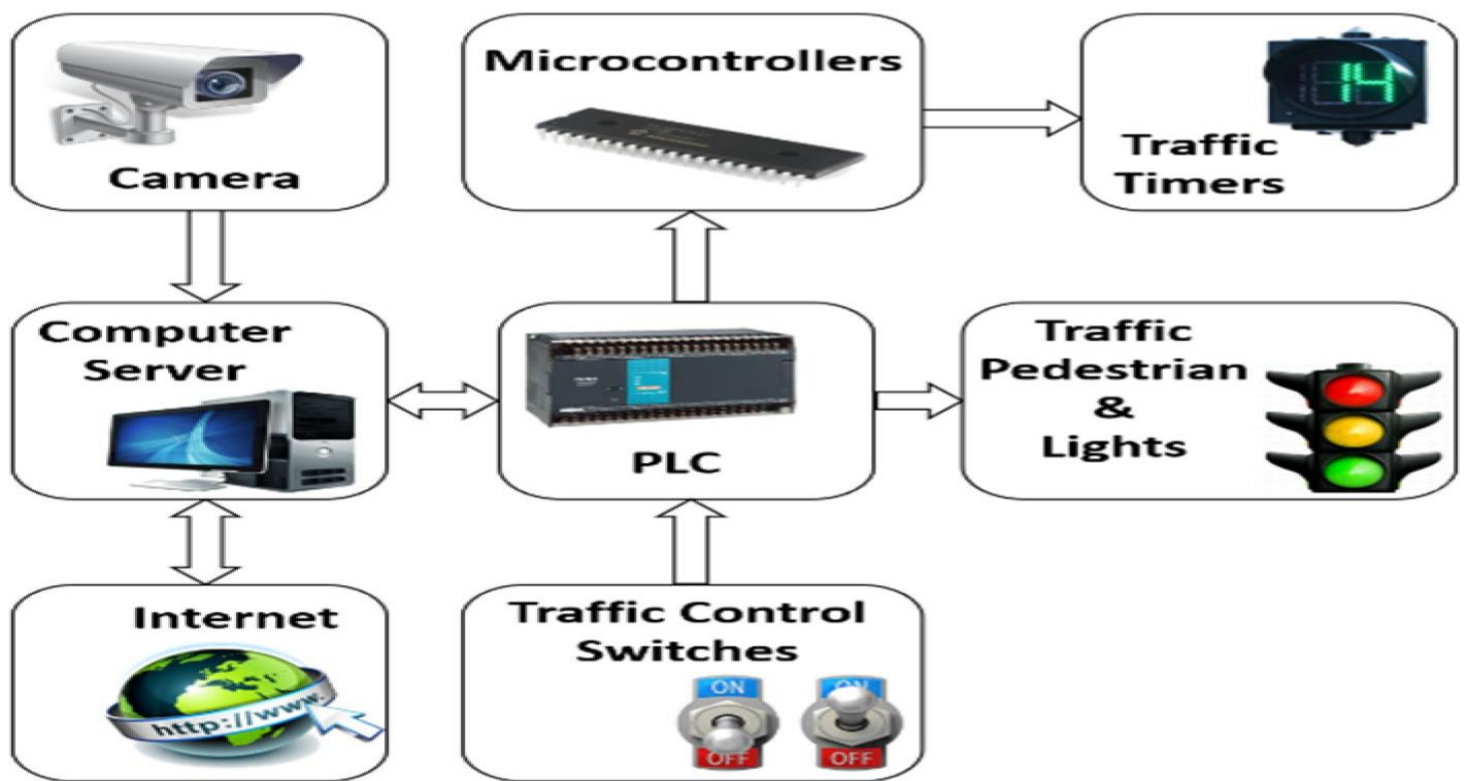
Based on the received input:

- **Low Density** → Short green signal duration
- **Medium Density** → Moderate signal timing
- **High Density** → Extended green signal duration

The system ensures that traffic signals are dynamically adjusted according to real-time conditions, improving traffic flow efficiency.

5. Output Module (Traffic Signals)

The final output is represented using LED indicators corresponding to traffic lights (Red, Yellow, Green). The ESP32 DevKit controls these LEDs to simulate real-world traffic signal behavior.



Hardware Components

The proposed smart traffic light control system is implemented using a set of low-cost and efficient hardware components. Each component plays a crucial role in enabling real-time image acquisition, processing, communication, and traffic signal control.

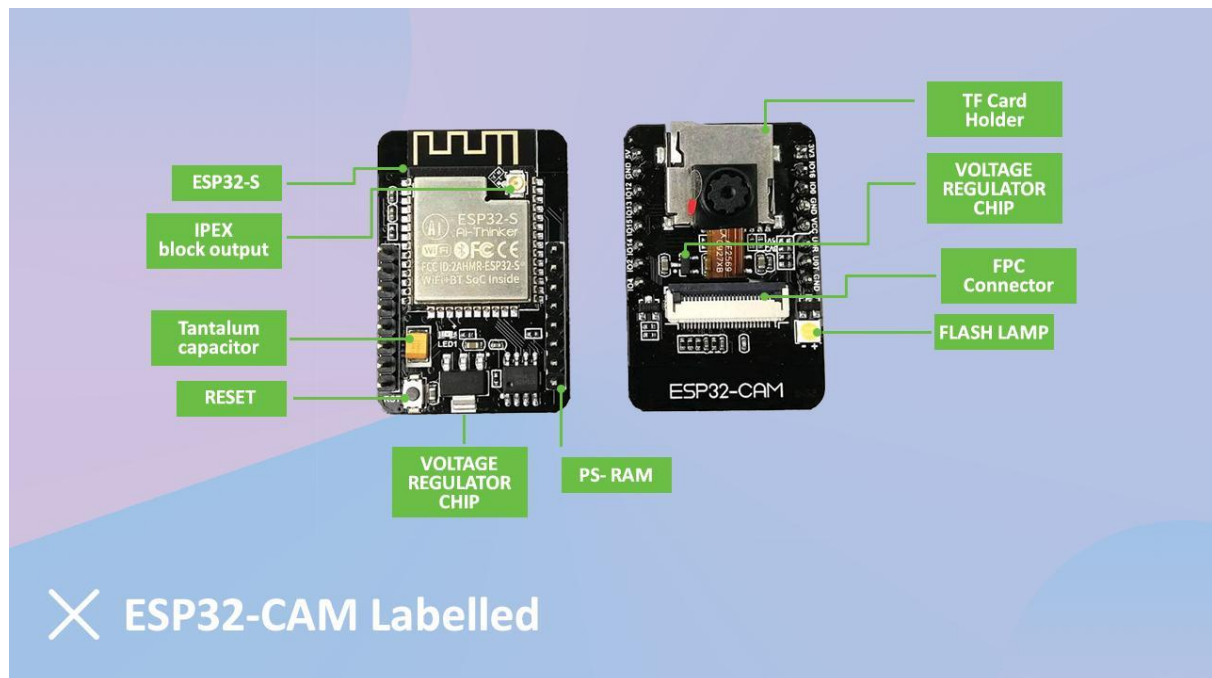
1. ESP32 DevKit

The ESP32 DevKit acts as the central control unit of the system. It is a powerful microcontroller with built-in WiFi and Bluetooth capabilities, making it suitable for IoT-based applications.

In this project, the ESP32 DevKit is responsible for:

- Receiving traffic density data from the ESP32-CAM via WiFi
- Processing the received information
- Controlling the traffic signal LEDs based on the detected density

Its high processing capability and wireless communication features make it ideal for real-time embedded applications.



2. ESP32-CAM

The ESP32-CAM module is used for image capture and on-device processing. It integrates a camera with a microcontroller, allowing both image acquisition and execution of TinyML models on a single device.

Functions of ESP32-CAM in this project:

- Captures real-time images of traffic
- Runs the trained TinyML model locally
- Classifies traffic density into low, medium, or high
- Sends classification results to the ESP32 DevKit

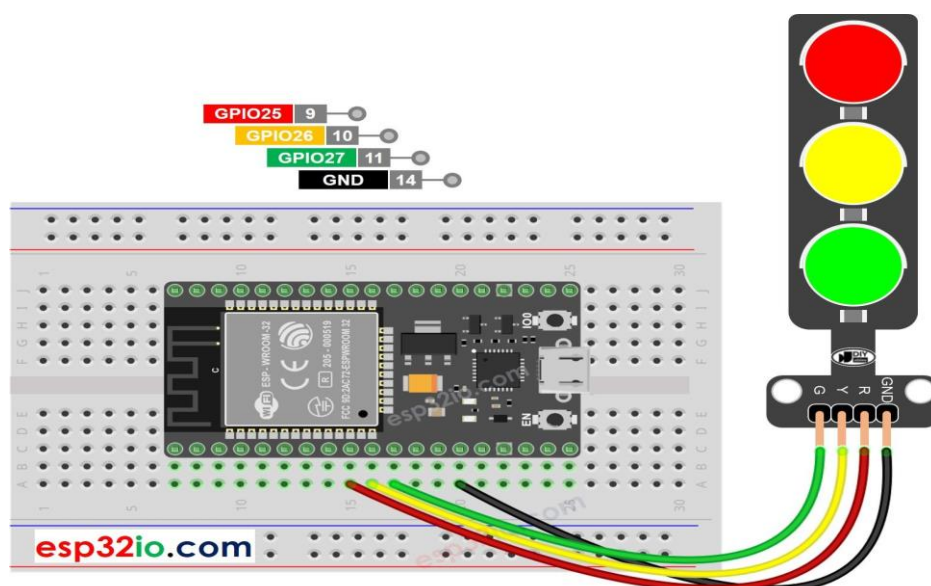
Its compact size and low power consumption make it suitable for edge-based vision applications.

3. LEDs (Traffic Signal Indicators)

Light Emitting Diodes (LEDs) are used to simulate traffic signals in the system. Three LEDs are used:

- Red LED → Stop signal
- Yellow LED → Transition signal
- Green LED → Go signal

The LEDs are controlled by the ESP32 DevKit and indicate traffic signal status based on the detected traffic density.



4. Resistors

Resistors are used along with LEDs to limit the current and prevent damage to the components. Typically, 220Ω resistors are used to ensure safe operation of LEDs.

They play a crucial role in protecting both the LEDs and the microcontroller from excessive current flow.

5. Power Supply

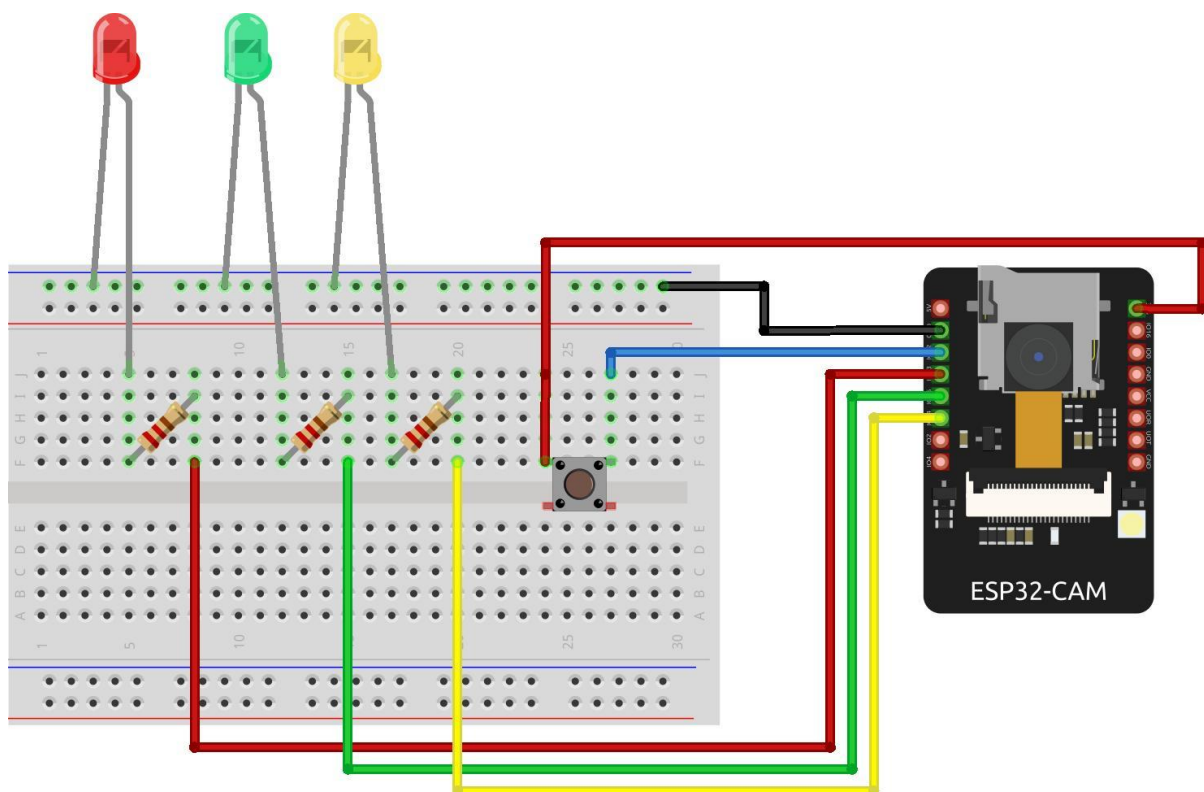
A stable power supply is essential for the proper functioning of the system. The ESP32 DevKit and ESP32-CAM can be powered using:

- USB power source
- External regulated 5V supply

In practical deployment, voltage regulation components such as boost converters may be used to ensure stable power delivery.

6. Connecting Wires and Breadboard

Connecting wires and a breadboard are used for prototyping and establishing connections between components. They allow easy testing, modification, and debugging of the circuit during development.



Software & Tools

The development of the proposed smart traffic light control system involves a combination of embedded programming tools, machine learning platforms, and communication technologies. These tools enable efficient implementation of TinyML-based traffic density classification and real-time system control.

1. Arduino IDE

The Arduino Integrated Development Environment (IDE) is used for writing, compiling, and uploading code to both the ESP32 DevKit and ESP32-CAM.

Key uses in this project:

- Writing Embedded C/C++ programs
- Uploading firmware to ESP32 boards
- Serial monitoring for debugging and testing

Its simplicity and wide community support make it suitable for rapid prototyping of embedded systems.

2. Edge Impulse

Edge Impulse is used for developing and deploying the TinyML model. It provides an end-to-end platform for data collection, model training, testing, and deployment on microcontrollers.

Role in this project:

- Collecting and labeling traffic image data
- Training a CNN model for density classification
- Optimizing the model for embedded deployment
- Exporting the model compatible with microcontrollers

It simplifies the process of implementing machine learning on low-power devices.

3. TensorFlow Lite for Microcontrollers

TensorFlow Lite for Microcontrollers is used to run the trained machine learning model on the ESP32-CAM. It enables on-device inference with minimal memory and processing requirements.

Functions:

- Executes the trained TinyML model

- Processes input images in real time
- Produces classification output (low, medium, high)

It is specifically designed for resource-constrained devices like microcontrollers.

4. Embedded C/C++ Programming

The system is implemented using Embedded C/C++, which is used to control hardware components and implement system logic.

Applications in the project:

- Handling GPIO operations for LED control
- Managing WiFi communication
- Integrating TinyML model inference
- Implementing decision-making logic

Embedded C provides low-level control and efficient execution for real-time applications.

5. WiFi and HTTP Communication

Wireless communication between the ESP32-CAM and ESP32 DevKit is achieved using WiFi and HTTP protocols.

Key functionalities:

- ESP32 DevKit acts as a WiFi Access Point
- ESP32-CAM connects to the network
- Data is transmitted using HTTP requests (GET/POST)
- Ensures real-time communication between modules

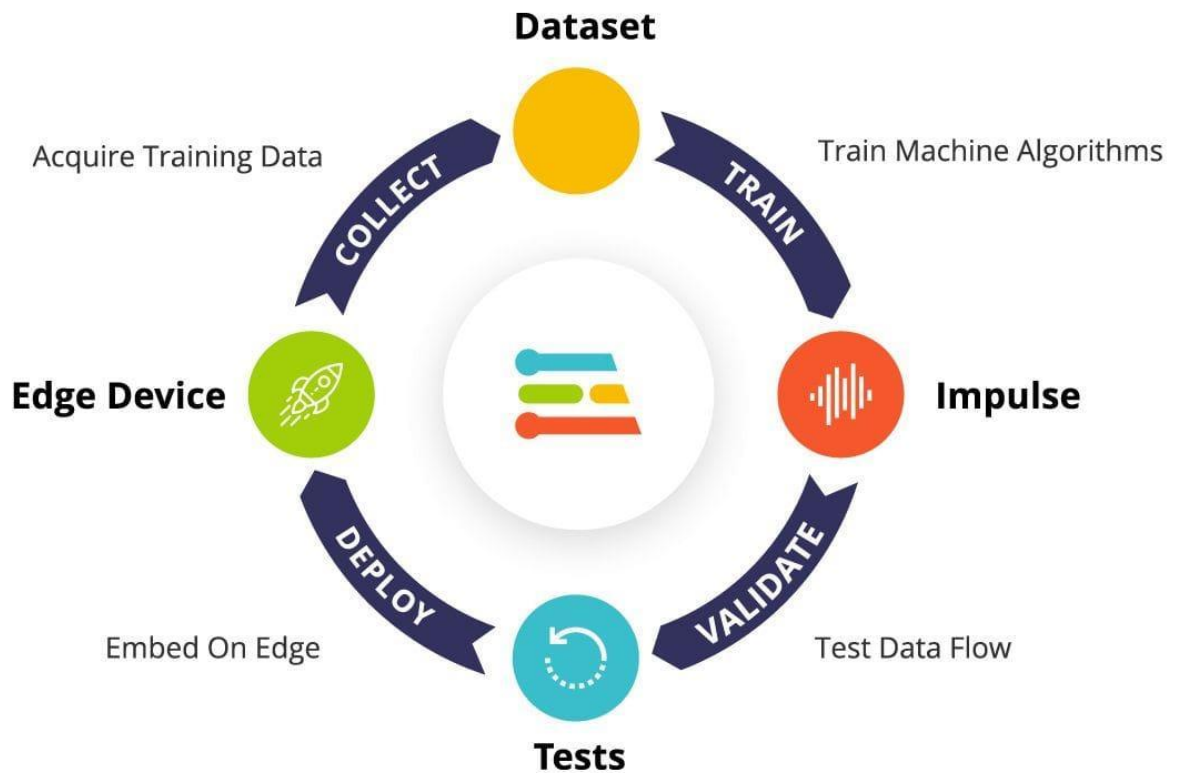
This enables seamless data transfer without requiring additional communication hardware.

6. Serial Monitor (Debugging Tool)

The Serial Monitor in Arduino IDE is used for debugging and monitoring system behavior.

Uses:

- Displaying traffic density results
- Monitoring communication status



. Methodology / Working

The proposed smart traffic light control system operates through a sequence of well-defined steps that enable real-time traffic density detection and adaptive signal control. The system integrates image acquisition, machine learning inference, wireless communication, and hardware control to achieve efficient traffic management.

Step 1: Image Capture

The process begins with the ESP32-CAM capturing real-time images of the traffic at an intersection. The camera is positioned in such a way that it provides a clear view of the road, allowing accurate observation of vehicle density.

Step 2: Image Preprocessing

Once the image is captured, it undergoes preprocessing to make it suitable for model input. This includes resizing the image, normalizing pixel values, and converting it into a format compatible with the TinyML model.

This step ensures that the input data is consistent and improves the accuracy of the model's predictions.

Step 3: Traffic Density Prediction (TinyML Inference)

The preprocessed image is then passed to the trained Convolutional Neural Network (CNN) model deployed on the ESP32-CAM using Edge Impulse and TensorFlow Lite for Microcontrollers.

The model analyzes the image and classifies traffic density into one of the following categories:

- Low traffic density
- Medium traffic density
- High traffic density

Along with the classification, the model also generates a confidence score indicating the reliability of the prediction.

Step 4: Data Transmission

After classification, the ESP32-CAM transmits the result (density level and confidence score) to the ESP32 DevKit using WiFi communication. HTTP protocol is used to send the data efficiently and reliably.

Step 5: Signal Decision Logic

The ESP32 DevKit receives the transmitted data and processes it to determine the appropriate traffic signal behavior. Based on the detected traffic density:

- **Low Density** → Short green signal duration
- **Medium Density** → Moderate signal timing
- **High Density** → Extended green signal duration

This dynamic adjustment ensures that traffic signals respond effectively to real-time conditions.

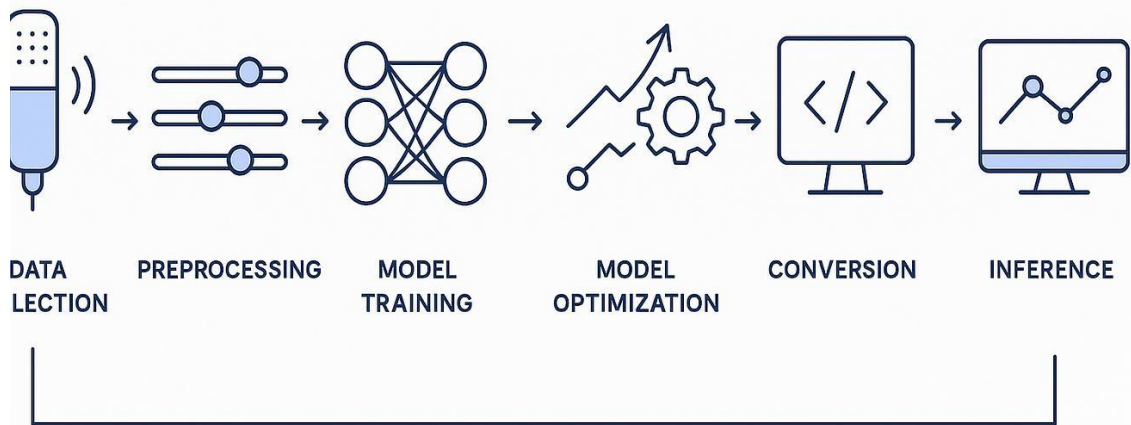
Step 6: Traffic Signal Control

Finally, the ESP32 DevKit controls the LED indicators representing traffic signals. The green LED is activated for a duration calculated based on the traffic density and confidence score. After the completion of the green signal duration, the system resets and prepares for the next cycle.

Step 7: Continuous Operation

The entire process operates in a continuous loop, allowing the system to constantly monitor traffic conditions and update signal timings accordingly. This ensures real-time responsiveness and efficient traffic flow management.

TinyML Pipeline



Advantages

The proposed TinyML-based smart traffic light control system offers several advantages over traditional traffic management systems.

One of the key advantages is **real-time adaptability**. Unlike conventional systems that operate on fixed timing, this system dynamically adjusts traffic signals based on actual traffic density, improving overall traffic flow.

Another major benefit is **low latency**, as the machine learning model runs directly on the ESP32-CAM. This eliminates delays associated with cloud-based processing and ensures faster decision-making.

The system is also **cost-effective**, as it uses affordable hardware components such as the ESP32 DevKit and does not require expensive infrastructure or servers.

Additionally, it provides **energy efficiency**, since TinyML models are optimized for low-power devices. This makes the system suitable for continuous operation without high power consumption.

Another important advantage is **data privacy and security**, as all processing is performed locally on the device without sending sensitive data to external servers.

Finally, the system is **scalable and flexible**, meaning it can be extended to multiple intersections or integrated into larger smart city frameworks.

Limitations

Despite its advantages, the system has certain limitations that need to be considered.

One limitation is the **restricted communication range**, as WiFi-based communication limits the operational distance between the camera module and the control unit.

Another limitation is **dependency on dataset quality**. The accuracy of traffic density classification depends on how well the model is trained. Poor or limited datasets may reduce prediction accuracy.

The system is also affected by **environmental conditions**, such as low lighting, shadows, or bad weather, which may impact image clarity and model performance.

Additionally, the current implementation is a **prototype-level system**, using LEDs to simulate traffic signals rather than controlling real-world traffic infrastructure.

Future Scope

The proposed system can be further enhanced in several ways to improve its performance and applicability.

One major improvement is the integration of **drone-based monitoring**, where the ESP32-CAM can be mounted on a drone to capture aerial traffic images for better coverage.

The system can also be upgraded with **advanced AI models**, such as vehicle detection and counting using deep learning techniques, to improve accuracy beyond simple density classification.

Another future enhancement is **cloud integration and IoT connectivity**, enabling centralized monitoring and control of multiple traffic intersections in a smart city environment.

The use of **long-range communication technologies** such as LoRa or 5G can also improve system scalability and coverage.

Finally, the system can be integrated with **real-world traffic infrastructure**, replacing LED indicators with actual traffic signals to enable deployment in urban environments.



Conclusion

The TinyML-based smart traffic light control system presented in this project successfully demonstrates an intelligent approach to traffic management using embedded systems and machine learning. By integrating real-time image capture, on-device inference, and wireless communication, the system is capable of dynamically adapting traffic signal behavior based on actual road conditions.

The use of the ESP32-CAM enables efficient image acquisition and real-time traffic density classification using a lightweight Convolutional Neural Network (CNN) model. The deployment of the model through Edge Impulse and execution using TensorFlow Lite for Microcontrollers ensures low-latency processing without reliance on cloud infrastructure.

The ESP32 DevKit effectively receives the classification results and controls the traffic signals accordingly. By adjusting the green signal duration based on traffic density, the system improves traffic flow, reduces waiting time, and minimizes fuel consumption.

Overall, the project highlights the practical application of TinyML in real-time embedded systems and demonstrates how edge intelligence can be used to build efficient, scalable, and cost-effective solutions for smart city applications. The system provides a strong foundation for future enhancements and real-world deployment.

References

The following resources were used during the development of this project:

1. Edge Impulse — Official documentation and tutorials
2. Arduino IDE — Programming and debugging tools
3. TensorFlow Lite for Microcontrollers — Model deployment framework
4. ESP32 DevKit — Hardware documentation
5. ESP32-CAM — Camera module reference and setup guides